

There are many other user contributed libraries to perform many more advanced operations such as handling 3D drawings, different hardware, animation, and so on. For more details about the available list of libraries, refer to: <http://processing.org/reference/libraries/>

Vega

Now let us look at Vega, another DSL you can use to create interactive data visualisation. It is built based on the concepts of the D3 and Protovis systems. It gives a wide range of visualisations such as pie/bar charts, timelines and choropleth, with complete control over data rendering and configuration. Vega is primarily targeted at designing Web-based visualisation, which can be rendered using HTML5 Canvas and Scalable Vector Graphics (SVG).

Vega specification

Vega specification is a simple JSON object used for configuring the various aspects of the data that needs to be rendered. Specification consists of the dataset to be used, the type of visualisation, the data rendering algorithm and other properties of a chart like the axis, scale, legend, etc. Visualisation is the top-level property in the specification and holds all other entities of configuration. Visualisation consists of a rectangular canvas and viewport, over which all visual elements will be added. The following is a simple visualisation configuration:

```
{
  "name": "demo",
  "width": 400,
  "height": 400,
  "padding": { "top": 5, "left": 20, "bottom": 5, "right": 20 }
}
```

Other major components of the visualisation section are discussed below.

Data

This section of the specification is to configure and define the data to load and its type. By default, Vega is configured to use a tabular data structure with collections of rows as records and columns as fields. Data can either be directly loaded through the *values* property or through *url* as shown below:

```
"data": [{"name": "temperature", "values": [244.85, 50.74,
2.95, 1.25, 44.04, 116.34]}]
"data": [{"name": "stats", "url": "data/playerStats.json",
"transform": [ ]}]
```

Apart from these basic configurations, it is also possible to specify the *format* of data to load (*csv*, *tsv*, *json*). The *transform* section is used to define how to render the data being loaded. *Data transforms* operations are executed over

the given dataset before rendering the visualisation. Its main purpose is to define the encoding mechanism and to filter, group, sort or define the custom formula. Each transform section should have *type* property, which defines the type of transformation to be performed and some set of properties. Some examples are shown below:

```
"transform": [
  { "type": "copy", "from": "data", "fields": [ "source",
  "target" ] }
  { "type": "filter", "test": "data.score > 100" }
  { "type": "formula", "field": "rectArea", "expr": "data.
length*data.width" }
]
```

Scales

A Scales section is used to convert the dataset loaded into a range of visual values such as colours, pixels and sizes, i.e., the bar's length in the bar chart, the area to colour in a pie chart, etc. We can define the type of scale to choose for visualising the data. Scale can be defined to use quantitative, ordinal, time or custom data.

```
"scales": [
  { "name": "x", "type": "ordinal", "range": "width",
  "domain": { "data": "table", "field": "data.year" },
  { "name": "y", "range": "height", "nice": true,
  "domain": { "data": "table", "field": "data.percent" } }
]
```

In the above example, the X-axis will use the ordinal scale, the domain specifies the data values to use - the field year will be used in the X-axis and the field per cent for Y-axis, while the range property defines the maximum value to plot. You can define and configure each attribute in plotting using different properties such as Category10 and Category20 for the colour, height, width and shapes of each group and the min-max values to plot.

Axes

This section defines various properties of the axis lines such as the title, label, ticks and lines to draw. You can configure to use major, minor and end type of tick marks.

```
"axes": [
  {
    "type": "y", "scale": "sectors", "tickSize": 0,
    "tickPadding": 0, "values": [ "A", "B", "C", "D", "E", "F" ]
  }
]
```

In the above example, we have configured the properties of the Y-axis to use a tick mark size of '0' and custom label values for the plot groups.